

Using Weisfeiler-Leman Features for Algorithm Selection in Constraint Optimisation

Anonymus ✉

Anonymus

Abstract

We have new features and they are decent enough.

2012 ACM Subject Classification Computing methodologies → Artificial intelligence

Keywords and phrases Constraint modelling, algorithm selection, machine learning, constraint solvers.

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

2 Background

3 Graph Conversion

Although seeing a combinatorial problem as a graph feels quite natural, how the graph should be structured is less obvious. Over the years, many different approaches have emerged: a lot of MIP-related tasks use a bipartite graph representation where a pair of variable/constraint nodes share an edge when the variable has a non-zero coefficient in the constraint [6]. Another possibility, often used in CP, is to represent the problem as a tripartite graph, where nodes are constraints, variables and possible values that variables can take. Similarly to the bipartite version, edges are present between variables and constraint nodes if a variable has a non-zero coefficient in the constraint. Furthermore, variable nodes also have edges that connect them to all possible values they can take [3]. Finally, Boisvert et al. [2] decide to decompose constraints into smaller components, and the final graph is akin to the abstract syntax tree of a program.

Our approach is inspired by that of Boisvert et al. [2], but, unlike them, we do not include domain values in our representation; furthermore, our graphs are directed instead of undirected. We have decided to use Boisvert et al.'s syntax tree-like representation as it allows us to decompose complex constraints into simpler elements that could be reused in different constraints with similar meanings.

In more details, we define five types of nodes:

- **Variable nodes:** representing the variables of the problem.
- **Solution node:** a single node for each problem that represents the type of problem (Minimisation, Maximisation or Satisfaction).
- **Parameter/Literal nodes:** representing the literal values of the problem (e.g. 1, 2, True, False, etc.).
- **Operator nodes:** representing arithmetic and functional operations between variables, parameters and other operators (e.g. +, ×, ÷ etc.) Operator nodes have a sub-category for the type of operator represented.
- **Constraint nodes:** representing relational predicates (e.g. =, ≠, <, ≤), logical operations (e.g. ∧, ∨, ¬), and global constraints (e.g. `all_different`, `table`). Constraint nodes also have a sub-category for the type of constraint the node represents.

All node and sub-node types we have defined can be seen in the appendix.

As previously mentioned, edges between nodes are directed: variables and parameter nodes have no incoming nodes and only point to the operators or constraints they take part in. There is only one exception: for optimisation problems, the variable containing the objective value to optimise has an



© Anonymus;

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

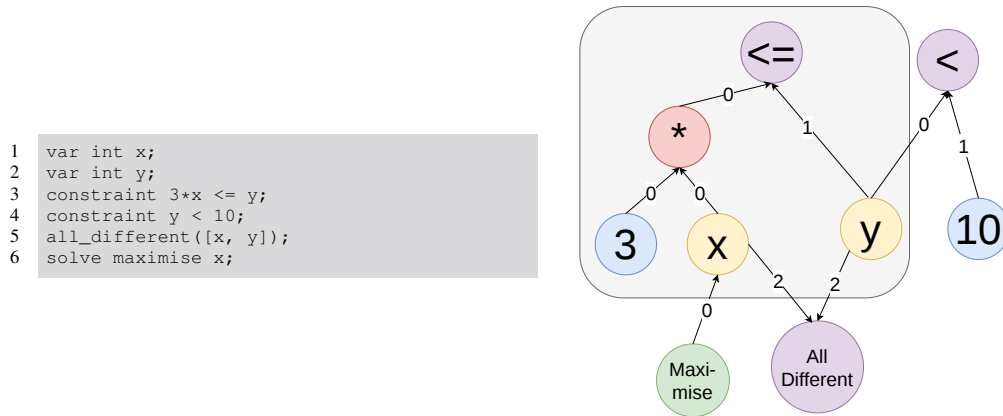
LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

XX:2 Using Weisfeiler-Leman Features for Algorithm Selection in Constraint Optimisation

incoming edge from the Solution node. Operator nodes can have both incoming and outgoing edges. Incoming edges come from variables, parameters and other operators that take part in the operation. Outgoing edges go to other operator nodes or to constraint nodes. Constraint nodes can also have edges to other constraint nodes when they appear as a sub-expression to other constraints. As an example, the expression $(x = 3) \wedge (y \neq 5)$, will have three constraint nodes ($=$, $\wedge$ and $\neq$) with $=$ and $\neq$ having exiting edges to $\wedge$.

Edges are typed as well; in particular, there are three types of edges:

- **Edge of type 0:** used to connect variables, parameters and operators to operators and constraints for which the order of the operands does not matter (e.g. sum or equality). 0 Edges are also used for variables, parameters and operators that fall on the left side of an operation or constraint where the order of the operands matters (e.g. subtraction or strictly less than). Type-0 edges are also used to connect the solution node to the objective variable in optimisation problems.
- **Edge of type 1:** Used to connect variables, parameters and operators to operators and constraints for which the order of the operands matters and the operands fall on the right side of the operation.
- **Edges of type 2:** Used to connect variables and parameters to global constraints. This helps to distinguish normal connections from normal and global constraints.



■ **Figure 1** Conversion of a constraint model (left) to a graph (right). Variables in yellow, Parameters in blue, Operators in red, Constraints in purple, and Solve in green. The area in gray represents the constraint $3 \times x \leq y$.

Figure 1 shows an example of how a simple constraint model can be converted using our conversion methodology. In many previous works, like [2], it has been argued that adding domain values to the graphs was essential to get a proper representation. The core of the argument was that, without including such elements, the conversion function would not be injective and, as a consequence, two different problems could fall into the same representation. In the instance in Figure 1, for example, our conversion would not be able to distinguish between an instance where the domain of the variable x is between 1 and 6 and one where it is between 2 and 5. However, adding domain information to the graph will result in a lot of extra nodes, and it could make the conversion too expensive for problems with big variable domains. Our primary objective is to use the converted graphs to extract features for algorithm selection purposes, and, as an example, an instance from the 2025 Minizinc challenge¹ (coming from the cgt problem with instance name cgt_12_h27_r0.05_s0.5_3) had an

¹ <https://www.minizinc.org/challenge/2025/results/>

objective function with a domain between -1.932.060 and 1.714.644, resulting in 3.646.705 total possible values just for one domain. The full instance would also require adding also other variables and their domains as well as constraints and operators. A graph with this many nodes would be intractable for our purposes.

4 Extracting Features Using The Weisfeiler-Leman Algorithm

Once we have defined how to convert any instance to a graph, we can use the Weisfeiler-Leman (WL) algorithm to extract features from the graphs.

4.1 The WL algorithm

The original purpose of the WL algorithm is to check if two graphs are isomorphic [7]. It does so by assigning a colour to each node on a graph and then iteratively refining it by aggregating its colour with the colours of the neighbouring nodes. The algorithm would keep refining the colours until either (a) the two graphs have a different colour set (meaning they are non-isomorphic) or (b) the refinement did not change the induced partitioning of the graphs (meaning that the graph are potentially isomorphic).

It is important to note that the WL algorithm does not guarantee correctness, and two graphs can be deemed isomorphic even though they are not. Typically, this happens when the two input graphs have the same node degree while still having a different structure.

It is also possible to generalise the WL algorithm to the k -WL algorithm where the colours are assigned to a k -tuple of nodes. In this sense, the standard WL algorithm is a special case of the k -WL algorithm where $k = 1$. At the growth of k , the k -WL algorithm gets much more powerful in distinguishing non-isomorphic graphs; however, its complexity also grows exponentially in k . Thus, for the purpose of this paper, we will only focus on the standard WL algorithm.

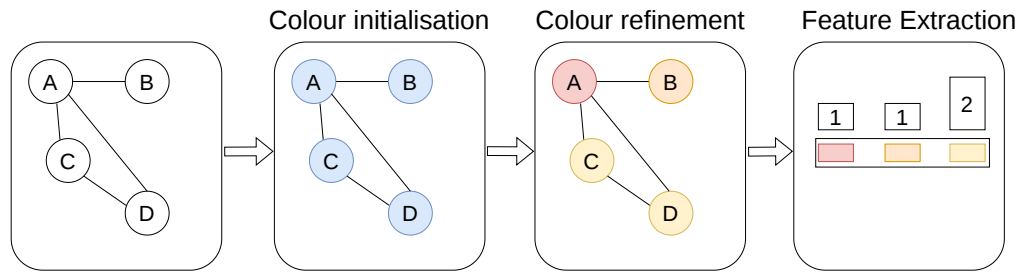
4.2 WL For Feature Extraction

To use the WL algorithm for feature extraction, we repeat the colour refinement process for a pre-defined number of steps on a single graph, then we extract features by creating a histogram where, for each colour, we count its occurrences in the graph. A depiction of the execution of the WL algorithm for feature extraction (with one aggregation step) can be seen in Figure 2. The WL features have been proven effective in many fields such as cheminformatics and computer vision. For a comprehensive review of the WL features for ML, we will refer to Morris et al. [8].

4.3 Enhancing Features' Expressiveness

The standard WL features are designed for graphs without any edge or node information. However, in our case, we have both node and edge-level information we can use to better represent the input graph. For this reason, we propose two extensions to the base algorithm:

- **Node-information extension:** To add node information into the extraction process, we can initialise the node colour with the hash of its type + subtype (e.g. a `Variable` node and a `Sum` node will be initialised with different colours). Formally, $c^{(0)}(v) \leftarrow \text{HASH}(\text{type}(v) || \text{subtype}(v))$ where $||$ denotes string concatenation.
- **Edge-information extension:** To add edge information into the extraction process, we can concatenate the edge type to the node colour during the aggregation step (e.g. in the example of Figure 2, if Node B was connected to node A via a type-0 edge, the aggregation of node B would



■ **Figure 2** Feature extraction process using the WL algorithm on a simple, undirected graph. The colour refinement is executed only once. At first, all nodes get the same colour, then node A aggregates its colour (blue) with its 3 neighbours’ colours (blue), getting, as a result, (blue, blue,blue,blue). This produces a new colour, red. C and D, having only two neighbours, get, as aggregation (blue, blue,blue), resulting in the new colour yellow and, finally, node B, having only one neighbour, aggregates to (blue, blue), getting as new colour orange. In the final steps, we count the number of occurrences of each colour, and we get 1 red, 1 orange and 2 yellow.

be: (blue, blue-0)). Formally, assuming a directed graph where edges flow towards v , its multiset update becomes $M(v) \leftarrow \{c^{(t-1)}(w) \mid \text{type}(w, v) \mid (w, v) \in E\}$.
Finally, we can use the fact that our graphs are directed and only update a node colour based on its incoming neighbours.

Using these extensions, we can now define four types of features: (i) WL, standard WL features without node or edge information, (ii) WLn, WL features enhanced via node-information extensions, (iii) WLe, WL features enhanced via edge-information extensions, and (iv) WLne, WL features enhanced with both edge and node-information.

4.4 The “Cut” Trick

In the WL algorithm, the colour of a node is highly dependent on its degree, and two nodes with different degrees, even when sharing the same original colour and the same type of neighbour nodes, will end up with different colours. This can be problematic for nodes representing operators and constraints where the number of connected sub-elements can vary. As an example, let’s examine the `all_different` global constraint: using the standard algorithm, `all_different([v1, v2, v3])` will get a totally different and uncorrelated colour to `all_different([v1, v2])` solely due to its degree. In practice, while preserving some differences between the two cases, we would still capture the fact that the two constraints are pretty similar.

To achieve this, we can virtually “cut” the connections between the nodes representing constraints and operators with a variable number of operands in them. We will refer to these nodes as cut nodes. In practice, during the aggregation steps, the cut nodes will not be updated since their incoming connections will be ignored. However, the cut nodes will still contribute to the update of the nodes they are connected to via their outgoing edges. All cut nodes can be seen in the appendix.

After having done so, and after the aggregation steps have been completed, each cut node’s colour can be updated by aggregating it with the set (not multiset) comprised of all its connected, original, node colours. By using a set rather than a multiset, nodes with different degrees but the same types of incoming neighbours will receive identical colours.

At this stage, we can correctly capture the fact that two constraints with the same operator, incoming edges but different degrees, have the same colour; however, we are completely missing any information regarding the degree of a node, information that we would still like to capture. To do so, we can add extra features after the colour histogram. Specifically, from each cut node, we can generate node pairs comprising the cut node and its incoming neighbours. Thus, each cut of degree n

will generate n pairs. After having generated all the pairs, we can count the occurrence of each pair in a graph to generate a histogram of pairs and concatenate it to the histogram of colours.

The cut trick can only be applied when node information is present, thus, only to WL_n and WL_{ne} features. We will call the features where the cut trick is applied WL_c if only the cut is applied to WL_n features and WL_{ce} if the cut is applied to WL_{ne} features. A formalisation of the WL_c features can be seen in Algorithm 1, and a graphical depiction of how the cut trick is applied can be seen in Figure 3.

■ **Algorithm 1** WL_c Algorithm for Feature Extraction. The WL_{ce} version will be different only in the aggregation step where also the edge information will be included.

Require: Graph $G = (V, E)$, iterations T , subset $V_{cut} \subseteq V$ of cut nodes

Ensure: Feature histograms of colours and pairs

```

1: Initialisation:  $c^{(0)}(v) \leftarrow \text{HASH}(\text{type}(v) \parallel \text{subtype}(v))$  for all  $v \in V$ 
2:  $t \leftarrow 0$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:   for each node  $v \in V$  do
6:     if  $v \notin V_{cut}$  then
7:        $M(v) \leftarrow \{c^{(t-1)}(w) \mid (w, v) \in E\}$ 
8:        $c^{(t)}(v) \leftarrow \text{HASH}(c^{(t-1)}(v), \text{sort}(M(v)))$ 
9:     end if
10:  end for
11: until  $t = T$ 
12:  $\text{Pairs} \leftarrow \{\}$  ▷ Initialize multiset for pairs histogram
13: for each node  $v \in V_{cut}$  do
14:    $S(v) \leftarrow \{c^{(0)}(w) \mid (w, v) \in E\}$  ▷ Set of initial colours (types) of incoming neighbours
15:    $c^{(final)}(v) \leftarrow \text{HASH}(c^{(T)}(v), \text{sort}(S(v)))$ 
16:   for each node  $w$  where  $(w, v) \in E$  do
17:      $\text{Pairs} \leftarrow \text{Pairs} \cup \{c^{(0)}(w), c^{(0)}(v)\}$ 
18:   end for
19: end for
20:  $c^{(final)}(v) \leftarrow c^{(T)}(v)$  for all  $v \in V \setminus V_{cut}$ 
21: return  $\text{Histogram}(c^{(final)}) \cup \text{Histogram}(\text{Pairs})$ 

```

4.5 Features, Characteristics, and Normalisation

As a consequence of our design choices, especially of having directed graphs, the extracted features have emerging properties that are worth noting. In particular, since variable and parameter edges have no incoming node (but for the variable connected to the objective function), the colours of those values never get updated. Thus, one value in the final feature set would represent the number of variables in the graph, and another the number of parameters in the graph.

To give some extra information about variables and parameters in the graph, besides their total count, we decided to add two extra features, one representing the average number of constraints/operators a variable takes part in and one representing the same but for parameters.

For similar reasons, most paths in the graph are quite shallow, especially after the cut. As an example, in the $x + 3 = y$ constraint, we will have 2 variable nodes (x , y), one parameter node (3), one operator node (+) and one constraint node (=). The operator node can be updated only once since the values of the parameter and variable nodes connected to it will never get updated. At the same time, the value of the constraint node can be updated at most twice: once at the first step when

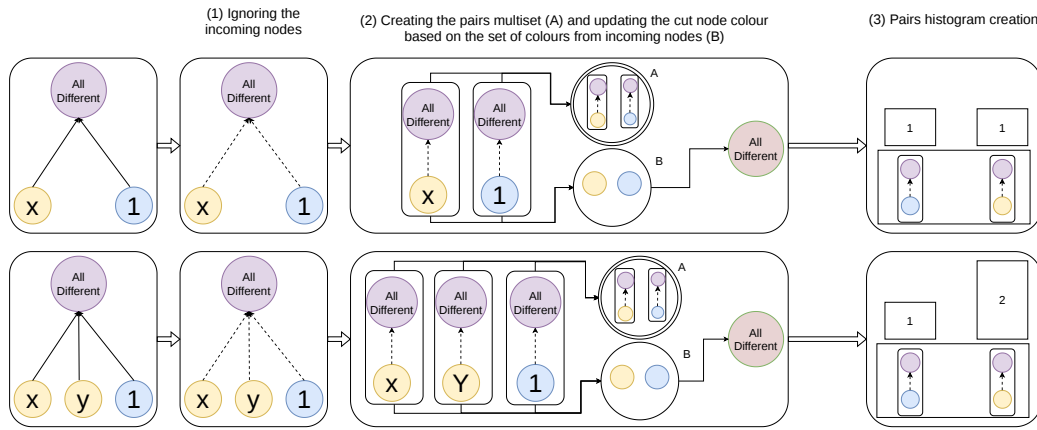


Figure 3 Application of the cut trick on two occurrences of the `all_different` constraint (top and bottom). Constraint nodes are in purple, variable nodes in yellow and parameter nodes in blue. First, we ignore colours from nodes with incoming edges (1), then we generate all pairs and update the colour of the `all_different` node by aggregating it with the set of all pairs (2) and then we generate the pairs histogram by counting all the occurrences of the pairs (3). In this example, since both constraints share the same type of incoming nodes, they will be coloured in the same way. However, having a different node degree will result in a different pair histogram.

161 it aggregates with the original `x` and `y` colours and once at the second step when it aggregates with
162 the updated `x` colour and the original, unmodified `y` colour.

163 Global constraints are always terminal nodes without any outgoing nodes; thus, as a consequence,
164 updating them only once, at the end of the standard WL steps, will result in the same output as
165 updating them during the standard WL steps.

166 Finally, once the final histograms are extracted from the graphs, we need to normalise the features.
167 This is a necessary step since the values in the histograms will grow proportionally to the size of the
168 graph. Thus, if not normalised, the produced histograms will be comparable only with those produced
169 by graphs of the same size, making it impossible to generalise.

170 We decided to normalise the colour histograms by dividing each value in the histogram by the
171 total count of nodes. Similarly, the pairs histogram is normalised by the total pairs count. This
172 normalisation step will generate a final feature set with values always smaller than or equal to 1.

173 4.6 Train and Test Feature Extraction Process

174 During the extraction process, different graph will generate different colours and only a subset of
175 those will be shared between different graphs. This will result in features sets with different histogram
176 sizes and with different colours in them. To train a ML model we need to standardise the histogram
177 making them share the same colours. To do so, we can collect all colours from all histograms and add
178 a 0 coefficient to all colours missing in a histogram.

179 During testing, if we encounter a new colour not present in the training data, we simply discard it
180 and ignore the update.

181 5 Experimental Results

182 To evaluate the proposed methodology, we implemented the graph conversion algorithm within the
183 FlatZinc modeling ecosystem. We selected 300 instances from the 2023, 2024, and 2025 MiniZinc
184 Challenges to construct three algorithm selection tasks: (i) maximizing the Borda count score across

three solvers, (ii) maximizing prediction accuracy for the same solver set, and (iii) predicting whether a parallel solver execution provides a significant performance gain over its sequential counterpart.

Our approach is compared against two baselines: a standard performance baseline and `fzn2feat` [1], a feature set specifically engineered for constraint programming algorithm selection.

Given the limited depth of our graph representations (see Section 4.5), we evaluate WL features using one and two aggregation steps.

For each task, we trained three models: a Support Vector Machine (SVM), a Random Forest, and a three-layer Multi-Layer Perceptron (MLP). The training pipeline followed a four-step protocol:

1. **Hyperparameter Optimization:** We performed a grid search over predefined configurations, evaluating each via mean 5-fold cross-validation.
2. **Model Selection:** We selected the highest-scoring configuration. Ties were broken by prioritizing simpler models to mitigate overfitting:
 - For SVM: lower regularization (C) and γ values, favoring the RBF kernel.
 - For Random Forest: fewer trees and shallower maximum depth.
 - For MLP: fewer training steps and a smaller α penalty.
3. **Dimensionality Reduction:** We evaluated the best configuration on reduced feature sets using Principal Component Analysis (PCA). Components were tested in the range $[20, \min(n_features, n_samples)]$ with a step size of 20.
4. **Final Feature Selection:** The feature set (original or PCA-transformed) yielding the highest cross-validation score was selected for final evaluation.

5.1 Borda Count Score Maximization

In this task, we compare three sequential solvers: `OR-Tools` [5], `Chuffed` [4], and `Cplex` [9] on 300 instances. Solvers were executed with the `-free-search` flag enabled² on an Intel Xeon Gold 6130 processor with a 20-minute timeout. Scores were assigned based on the Borda count strategy used in the MiniZinc Challenge.³ We framed this as a classification problem where the target label is the solver achieving the highest score.

We excluded one trivially unsatisfiable instance from which features could not be extracted, as well as instances where no solver reached a solution. The final dataset comprised 289 instances. Evaluation consisted of 10 repetitions of 5-fold cross-validation (50 total trials). We employed `StratifiedKFold` from `Scikit-learn`⁴, stratifying based on the performance gap between the Virtual Best Solver (VBS) and the worst-performing solver in the portfolio.

Figure 4 illustrates the performance of SVM models trained on WL-based features. While all feature sets generally outperformed the baseline, incorporating node information (`WLn`) yielded a significant performance increase over standard WL features. Conversely, edge information (`WLe`) provided no substantial predictive signal. Combining node and edge attributes (`WLne`) did not further improve results beyond the node-only baseline.

Notably, features derived from the cut-based representation (`WLc` and `WLce`) significantly outperformed all other variants. The number of aggregation steps had negligible impact on final performance across all feature sets. Although `WLc-1` was the only configuration to consistently exceed the baseline (> 1), the performance differences between various cut-based features were not statistically significant, likely due to high variance inherent in the task. Finally, Figure 5 demonstrates that all cut-based models statistically outperformed the `fzn2feat` baseline.

² https://docs.minizinc.dev/en/stable/command_line.html#cmdoption-f

³ <https://www.minizinc.org/challenge/2025/>

⁴ https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

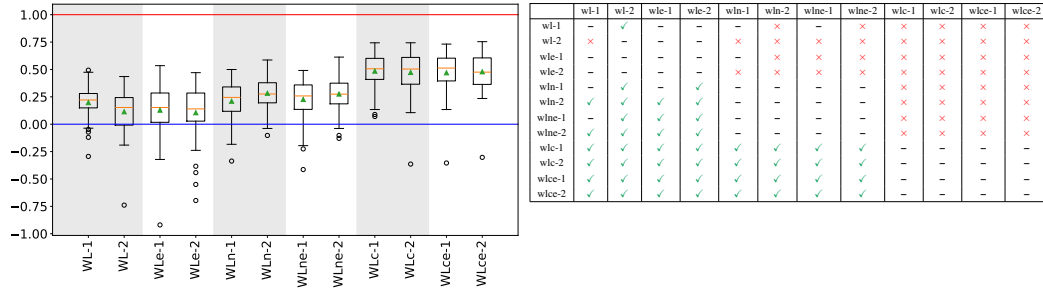


Figure 4 Performance results for SVM models trained on WL-based features for the Borda count maximization task. Scores (left) are normalized as $S_{norm} = \frac{S - S_{sbs}}{S_{vbs} - S_{sbs}}$, where S is the model score, S_{sbs} is the Single Best Solver, and S_{vbs} is the Virtual Best Solver. The statistical significance table (right) indicates results compared to the baseline using a Wilcoxon signed-rank test ($\alpha = 0.05$). Symbols: $\times$ denotes significantly worse performance ($p < 0.05$), $\checkmark$ denotes significantly better performance ($p < 0.05$), and $-$ indicates no statistically significant difference.

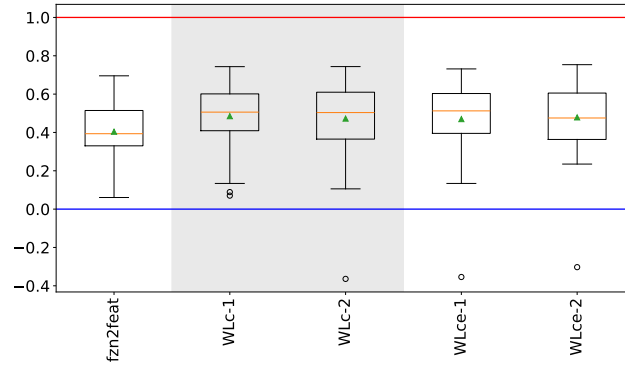


Figure 5 Comparative performance of cut-based features (WLc/WLce) against the `fzn2feat` baseline for the Borda count maximization task for SMV models. Scores are normalized relative to the Single Best Solver and Virtual Best Solver. Statistical significance was determined via a Wilcoxon signed-rank test; all cut-based variants outperformed `fzn2feat` with $p < 0.05$.

Since Random Forest and MLP classifiers exhibit performance trends similar to those of the SVM on WL-based features, we provide their comprehensive comparisons in the Appendix. We will now focus only on the performance of cut-based features relative to the `fzn2feat` baseline for these models.

Figure 6 illustrates the comparative performance of Random Forest and MLP models. For the Random Forest, `fzn2feat` achieves marginally higher scores than the cut-based features with the exception of `WLce-2` which still presents slightly higher mean and median scores, though these differences are not statistically significant ($p > 0.5$). In the MLP evaluation, only `WLc-2` significantly outperforms `fzn2feat` ($p = 0.04$), while the improvement for `WLc-1` is marginal and falls outside the significance threshold ($p = 0.054$).

Finally, we conducted a cross-model comparison between the best-performing `fzn2feat` configuration (Random Forest) and the best cut-based configuration (SVM). No statistically significant difference was observed, with the lowest recorded p -value being 0.33. However, it is noteworthy that all cut-based feature sets yielded higher mean and median scores than `fzn2feat` across these comparisons.

Finally, it is noteworthy to point out that we deliberately decided to exclude the feature extraction

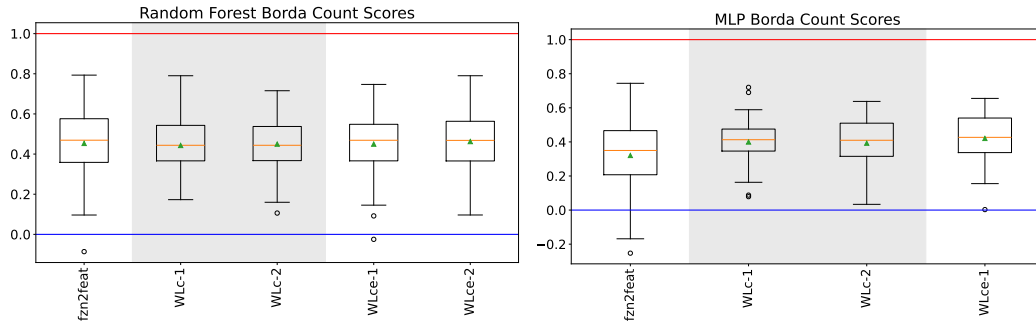


Figure 6 Comparative performance of cut-based features (WLC/WLce) against the `fzn2feat` baseline for the Borda count maximization task. Scores are normalized relative to the Single Best Solver and Virtual Best Solver. Results on the left are obtained by training Random Forest models while those on the right are obtained by training MLPs.

cost from the results since our current implementation of the feature extraction isn't well optimised and its performance are sub-optimal. A more detailed discussion about it can be found in Section ??

5.2 Algorithm Selection with Accuracy Maximisation

The setup for this task was the same as for the borda count maximisation one (in Section 5.1) with the only difference being the objective of the task which changes from maximising the score of the portfolio to maximising predictive accuracy. This change will allow us to use different statistical tests such as the McNemar test [10]. In this task, the baseline to beat is the majority classifier which always predicts the most frequent label.

The experimental configuration for this task mirrors the Borda count maximization setup (Section 5.1), with the objective shifted to maximizing predictive accuracy. This shift allows for the application of additional statistical evaluations, specifically the McNemar test [10], to assess the significance of differences in classifier performance. In this context, the baseline for comparison is the ZeroR (majority) classifier, which consistently predicts the most frequent class label in the training set.

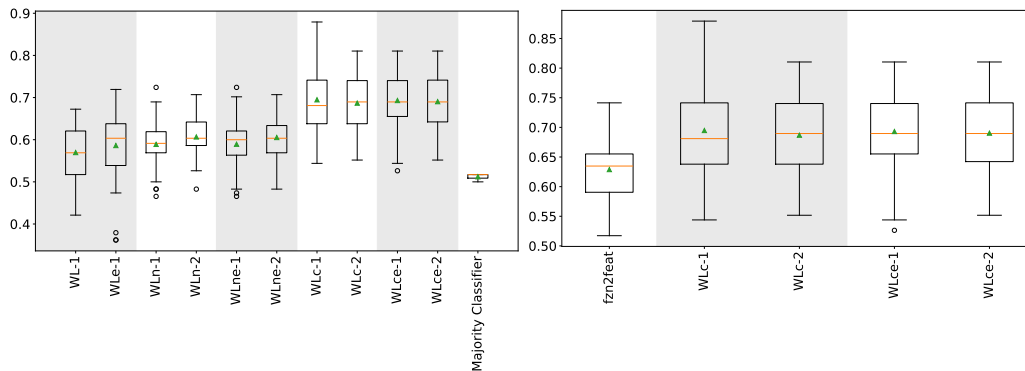
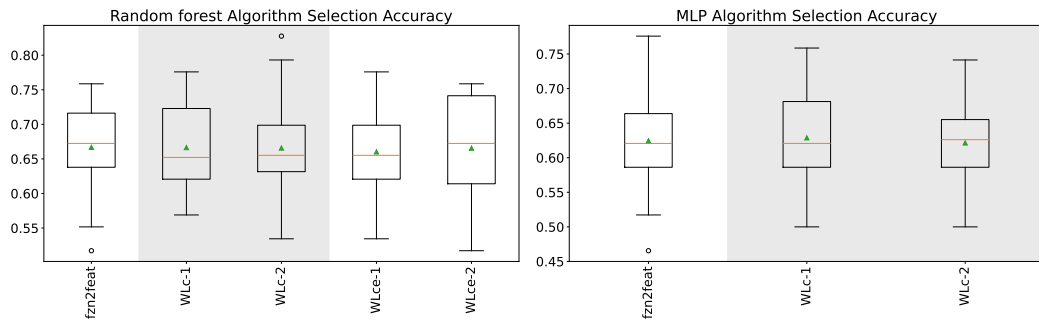


Figure 7 Predictive accuracy for SVM models trained on WL-based features (Left). Statistical significance relative to the majority classifier baseline is determined via Wilcoxon signed-rank and McNemar tests ($\alpha = 0.05$). And comparative accuracy of SVM models trained on cut-based features versus `fzn2feat` (Right). Significance was evaluated using McNemar and Wilcoxon signed-rank tests ($p < 0.01$).

Consistent with the observations in Section 5.1, the comparisons of WL-based features for SVM models (Figure 7, right) indicate that cut-based features significantly outperform alternative variants.



■ **Figure 8** Comparative performance of cut-based features (WLC/WLce) against `fzn2feat`. Results on the left are obtained by training Random Forest models while those on the right are obtained by training MLPs.

However, for the accuracy maximization task, the marginal benefit of incorporating node information is less pronounced than in the Borda count task. Regardless, all feature configurations demonstrate a statistically significant improvement over the majority classifier baseline.

When compared against the `fzn2feat` baseline (Figure 7, left), cut-based features exhibit significantly higher accuracy. This performance gain is confirmed by both McNemar and Wilcoxon signed-rank tests, with p -values below 0.01. Conversely, inter-model comparisons between the various cut-based feature sets do not yield statistically significant differences.

Figure 8 shows the performance of Random Forest and MLP-based classifiers trained with cut-based and `fzn2feat` features. While `fzn2feat` features perform slightly better on the Random Forest classifier, in no case there is statistical significance both with McNemar and Wilcoxon signed-rank tests.

When comparing the best `fzn2feat` classifier with the best cut-based ones, both WLC-1 and WLC-2-based models are significantly better using the McNemar test while only the WLC-1-based model is better using the Wilcoxon signed-rank tests.

References

- Roberto Amadini, Maurizio Gabbriellini, and Jacopo Mauro. An enhanced features extractor for a portfolio of constraint solvers. In *Proceedings of the 29th annual ACM symposium on applied computing*. 2014.
- Léo Boisvert, Hélène Verhaeghe, and Quentin Cappart. Towards a generic representation of combinatorial problems for learning-based approaches. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 99–108. Springer, 2024.
- Quentin Cappart, Didier Chételat, Elias B Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.
- Geoffrey Chu, Peter J. Stuckey, Andreas Schutt, Thorsten Ehlers, and Kathryn Francis Graeme Gange. Chuffed: The chuffed cp solver.
- Frédéric Didier, Laurent Perron, Sarah Mohajeri, Steven Agostino Gay, Thibaut Cuvelier, and Vincent Furnon. Or-tools’ vehicle routing solver: a generic constraint-programming solver with heuristic search for routing problems. 2023.
- Elias B Khalil, Christopher Morris, and Andrea Lodi. Mip-gnn: A data-driven framework for guiding combinatorial solvers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 10219–10227, 2022.
- Sandra Kiefer. *Power and limits of the Weisfeiler-Leman algorithm*. PhD thesis, Dissertation, RWTH Aachen University, 2020, 2020.

- 291 **8** Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M Kriege, Martin Grohe, Matthias
292 Fey, and Karsten Borgwardt. Weisfeiler and leman go machine learning: The story so far. *Journal of*
293 *Machine Learning Research*, 24(333):1–59, 2023.
- 294 **9** Stefan Nickel. *Decision optimization with IBM ILOG CPLEX optimization studio*. Springer, Berlin/Heidel-
295 berg; Germany, 2022.
- 296 **10** Pembury Smith, Q R Matilda, and Graeme D Ruxton. Effective use of the McNemar test. *Behavioral*
297 *Ecology and Sociobiology*, 74, 2020.